```
/*
 * Oktobar - 2021
 *
 * Koristeći WCF napisati čet aplikaciju.
 * Svaki korisnik pri startovanju klijenta registruje svoj nadimak.
 * Ukoliko se registruje isti nadimak, smatrati da je prethodna sesija prestala da važi.
 * Klijent pod svojim nadimkom šalje poruku drugom klijentu tako što navede njegov nadimak
 * ili šalje poruku svima tako što za nadimak korisnika setuje rezervisanu reč SVI.
 * Poruke namenjene svima stižu samo onim korisnicima koji su trenutno aktivni.
 * Korisnik može da zatraži istoriju primljenih poruka za traženi period -> MessageHistory (DateTime from, DateTime to).
 * Neophodno je da korisnik po pozivu ove metode prikaže sve poruke koje je dobio od servisa sa sledećim parametrima:
 * ko je poslao, kad je poslata, sadržaj poruke i da li je poruka namenjena samo njemu ili svima.
 */

namespace WCF.Contracts
{
    [ServiceContract(CallbackContract = typeof(IChatCallback), SessionMode = SessionMode.Required)]
    public interface IChatService
    {
        [OperationContract]
        void RegisterUser(string username);

        [OperationContract]
        void SendMessage(string sender, string recipient, string message);

        [OperationContract]
        List<Message> GetMessageHistory(DateTime from, DateTime to);
    }

    public interface IChatCallback
    {
        [OperationContract(IsOneWay = true)]
        void ReceiveMessage(string sender, string message);
    }

    [DataContract]
    public class Message
    {
        [DataMember]
        public string Sender { get; set; }

        [DataMember]
        public string Recipient { get; set; }

        [DataMember]
        public string Content { get; set; }

        [DataMember]
        public DateTime Timestamp { get; set; }

        [DataMember]
        public bool IsBroadcast { get; set; }
    }

    public class MessageHistory
    {
        private static MessageHistory instance;
        private readonly List<Message> messages;

        private MessageHistory()
        {
            messages = new List<Message>();
        }

        public static MessageHistory Instance
        {
            get
            {
                if (instance == null)
                {
                    instance = new MessageHistory();
                }
                return instance;
            }
        }

        public void AddMessage(Message message)
        {
            messages.Add(message);
        }

        public List<Message> GetMessages(DateTime from, DateTime to, String username)
        {
            return messages.Where(message => (message.Timestamp >= from && message.Timestamp <= to) && ((message.Recipient == username) || (message.IsBroadcast && message.Sender !=
                                                                                                                                                                  username))).ToList();
        }
    }

}

namespace WCF.Services
{
    [ServiceBehavior(InstanceContextMode = InstanceContextMode.PerSession)]
    public class ChatService : IChatService
    {
        private string _username;
        private IChatCallback _callback;
        private static readonly List<ChatService> _activeUsers = new List<ChatService>();
        private static readonly MessageHistory _messageHistory = MessageHistory.Instance;

        public void RegisterUser(string username)
        {
            _username = username;
            _callback = OperationContext.Current.GetCallbackChannel<IChatCallback>();

            // Remove any existing sessions for the same username
            _activeUsers.RemoveAll(user => user._username == username);
            _activeUsers.Add(this);
        }

        public void SendMessage(string sender, string recipient, string message)
        {
            _messageHistory.AddMessage(new Message
            {
                Sender = sender,
                Recipient = recipient,
                Content = message,
                Timestamp = DateTime.Now,
                IsBroadcast = recipient == "SVI"
            });
```

modifikacija broj 1

ako se traži u zadatku da sesija za Fallback username još uvek važi onda zakomentarišeš ovaj deo i napravi dole modifikaciju broj 2

```
            if (recipient == "SVI")
            {
                _activeUsers.ForEach(user =>
                {
                    if (user._username != sender) // Exclude sender from receiving their own message
                    {
                        user._callback.ReceiveMessage(sender, message);
                    }
                });
            }
            else
            {
                var recipientUser = _activeUsers.Find(user => user._username == recipient);
                recipientUser?._callback.ReceiveMessage(sender, message);
            }
        }

        public List<Message> GetMessageHistory(DateTime from, DateTime to)
        {
           return _messageHistory.GetMessages(from, to, _username);
        }
    }
}
```

Modifikacija broj (2)

1. zakomentarisati ceo ovaj deo

2. var recipientUsers = _activeUsers.FindAll(
   user => user._username == recipient);
   if (recipientUsers != null && recipientUsers.Count > 0)
   {
   foreach (var user in recipientUsers)
   {
   user._callback.ReceiveMessage(sender, message);
   }
   }

```
// Web.config
<configuration>
 <system.serviceModel>
  <services>
   <service name="WCF.Services.ChatService">
    <endpoint address="/Services/ChatService.svc" binding="wsDualHttpBinding" contract="WCF.Contracts.IChatService" />
   </service>
  </services>
 </system.serviceModel>
</configuration>
```

```
// App.config
<configuration>
    <system.serviceModel>
        <bindings>
            <wsDualHttpBinding>
                <binding name="WSDualHttpBinding_IChatService" />
            </wsDualHttpBinding>
        </bindings>
        <client>
            <endpoint address="http://localhost:56530/Services/ChatService.svc/Services/ChatService.svc"
                binding="wsDualHttpBinding" bindingConfiguration="WSDualHttpBinding_IChatService"
                contract="ChatServiceReference.IChatService" name="WSDualHttpBinding_IChatService">
            </endpoint>
        </client>
    </system.serviceModel>
</configuration>
```

dovoljno samo ovo

```
namespace ClientConsoleApp
{

    public class ChatCallback : IChatServiceCallback
    {
        public void ReceiveMessage(string sender, string message)
        {
            Console.WriteLine($"New message from: {sender}\nMessage: {message}");
        }
    }
    internal class Oktobar2021
    {
        private static ChatServiceClient chatServiceClient;

        public Oktobar2021()
        {
            Start();
        }

        public void Start()
        {
            ChatCallback callback = new ChatCallback();
            InstanceContext instanceContext = new InstanceContext(callback);

            try
            {
                using (chatServiceClient = new ChatServiceClient(instanceContext))
                {
                    Console.Write("Enter your username: ");
                    string username = Console.ReadLine();

                    try
                    {
                        chatServiceClient.RegisterUser(username.Trim());

                        Console.WriteLine("Type 'EXIT' to exit.");
                        Console.WriteLine("Type 'SVI' to send a broadcast message.");
                        Console.WriteLine("Format for private message: [Message] -[RecipientUsername]");
                        Console.WriteLine("Type 'HISTORY' to show the message history for specific date range");

                        while (true)
                        {
                            string input = Console.ReadLine();

                            if (input.ToLower().Trim() == "exit")
                                break;

                            if (input.ToLower().Trim() == "svi")
                            {
                                Console.Write("Enter your broadcast message: ");
                                string message = Console.ReadLine();
                                chatServiceClient.SendMessage(username, "SVI", message);
                            }
                            else if (input.ToLower().Trim() == "history")
                            {
                                Console.Write("Enter the start date (yyyy-MM-dd HH:mm:ss): ");
                                if (DateTime.TryParse(Console.ReadLine(), out DateTime from))
                                {
                                    Console.Write("Enter the end date (yyyy-MM-dd HH:mm:ss): ");
                                    if (DateTime.TryParse(Console.ReadLine(), out DateTime to))
                                    {
                                        Message[] history = chatServiceClient.GetMessageHistory(from, to);
                                        if (history.Length > 0)
                                        {
                                            foreach (Message message in history)
                                            {
                                                Console.WriteLine($"From: {message.Sender}, To: {message.Recipient}, Timestamp: {message.Timestamp}, Content: {message.Content}, Broadca
                                            }
```

```
                                    }
                                    else
                                    {
                                        Console.WriteLine("No message history found.");
                                    }
                                }
                                else
                                {
                                    Console.WriteLine("Invalid end date format.");
                                }
                            }
                            else
                            {
                                Console.WriteLine("Invalid start date format.");
                            }
                        }
                        else if (input.Contains("-"))
                        {
                            string[] parts = input.Split('-');
                            if (parts.Length == 2)
                            {
                                string recipient = parts[1].Trim();
                                string message = parts[0].Trim();
                                chatServiceClient.SendMessage(username, recipient, message);
                            }
                            else
                            {
                                Console.WriteLine("Invalid input format. Please use the following formats:");
                                Console.WriteLine("For private message: [Message] -[RecipientUsername]");
                            }
                        }
                        else
                        {
                            Console.WriteLine("Invalid input. Type 'EXIT', 'SVI', 'HISTORY', or use the specified message formats: [Message] -[RecipientUsername]");
                        }
                    }
                }
                catch (EndpointNotFoundException)
                {
                    Console.WriteLine("Error connecting to the service. Please ensure the service is running.");
                    Thread.Sleep(2000);
                }
                catch (Exception ex)
                {
                    Console.WriteLine("An error occurred: " + ex.Message);
                    Thread.Sleep(2000);
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("An error occurred: " + ex.Message);
            Thread.Sleep(2000);
        }
    }
}
}

namespace ClientConsoleApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            new Oktobar2021();
        }
    }
}
```